

FRONTEND INTEGRATION GUIDE

LEX AI Chat APIs

Working contract, request/response shapes, chat widget flow, lawyer metering, timeouts, and UI rules for the NexusLexis legal assistant.



Document ID NL-FE-LEX-001

Version 1.5 · Date 11 August 2026

Owner NexusLexis Backend

Applies to Client chat widget only

PRODUCTION BASE

<https://nexus-lexis-backend-ql8w.vercel.app/api/v1/lex>

Contents

- 1. Purpose & environment
- 2. Architecture — REST only in production
- 3. Client chat widget flow
- 4. POST /chat/ contract
- 5. Reply decision flow
- 6. Length, timing & timeouts
- 7. Chat history — New chat + sidebar (server)
- 8. UI contract & errors
- 9. Sample TypeScript
- 10. Smoke test & acceptance checklist
- 11. API index

START HERE Production chat is REST JSON. Do not open WebSockets. Send one `session_key` per thread. History is stored on the Main API (POST/GET/DELETE /sessions/). Client widget only.

1. Purpose & environment

LEX is the NexusLexis legal assistant. It answers Pakistani law questions in English, Urdu, or Roman Urdu. It is not a licensed advocate — the widget must always show a disclaimer. This document is the frontend working contract.

VITE_LEX_API_BASE_URL=https://nexus-lexis-backend-ql8w.vercel.app/api/v1/lex
VITE_API_BASE_URL=https://nexus-lexis-backend-ql8w.vercel.app/api/v2

Who	Call
Public / client widget	{VITE_LEX_API_BASE_URL}/chat/

DO NOT Do not set VITE_LEX_WS_URL for production. ws://.../api/lex/ws exists only on local Main API. Vercel serverless has no sockets.

2. Architecture — REST only in production

The browser always talks to the Main API on Vercel. Production is Vercel only — Main + Auth. LEX chat runs inline on that same Main API (Node + Gemini).

Frontend
POST /api/v1/lex/chat/ { message, session_key }
 ■
 ▼
Main API (Vercel)
nexus-lexis-backend-ql8w.vercel.app
LEX_MODE=inline → Node + Gemini + question bank

Auth API (Vercel)
nexus-lexis-backend-45v4.vercel.app

Service	Production URL
Main API + LEX	https://nexus-lexis-backend-ql8w.vercel.app

Service	Production URL
Auth API	https://nexus-lexis-backend-45v4.vercel.app

3. Client chat widget flow

- On open: show a local greeting, or send Hello.
- Create one `session_key` per thread: `session_${Date.now()}_${rand}`.
- Optimistic user bubble → typing indicator → POST /chat/.
- Render response. If `language === "UR"`, set `dir="rtl"`.
- If `show_lawyer === true`, show CTA → /find-a-lawyer.
- Keep `messages[]` in React state; persist to `localStorage` so refresh does not wipe the thread.

Auth: public chat does not require a JWT. Sending a logged-in token is fine (ignored today).

4. POST /chat/ contract

4.1 Request

```
POST https://nexus-lexis-backend-ql8w.vercel.app/api/v1/lex/chat/
Content-Type: application/json
```

```
{
  "message": "How do I register a company in Pakistan?",
  "session_key": "session_1723280000000_ab12"
}
```

Field	Required	Notes
message	Yes	Non-empty. Empty → 400 { "error": "Message is required" }
session_key	Recommended	Stable id for this conversation thread

4.2 Success (200)

```
{
  "response": "To register a private limited company in Pakistan...",
  "language": "EN",
  "register": "PLAIN",
  "show_lawyer": false
}
```

Field	Type	Frontend use
response	string	Bot bubble. Wrap as paragraphs; no HTML from server.
language	EN UR	UR → RTL + Urdu-capable font on that bubble
register	PLAIN LEGAL	Optional “Legal terms” badge
show_lawyer	boolean	true → Find a Lawyer CTA

5. Reply decision flow

Same POST /chat/ for every message. LEX decides internally — the frontend always renders response.

User asks a question

-
- ▼
- 1. Introductory? (Hi / Salam / Who are you / What is LEX?)
 - yes → canned intro reply. STOP. No LLM. No bank.
 - no
- ▼
- 2. Law-related?
 - no → "I can only answer law-related questions." STOP. No LLM. No bank.
 - yes
- ▼
- 3. Search Question Bank
 - TF-IDF on the verified sheet (fast)
 - Embeddings of bank questions (when background index is ready)
 Match user question ↔ bank Q&A
 -
 - HIT → 4a. Pass verified Q+A into LLM.
 - LLM only forms sentence structure –
 - does not invent facts beyond the sheet.
 -
 - MISS → 4b. Connect to LLM (Gemini) directly for general Pakistani law.

Step	Condition	What LEX returns	LLM?
1 Intro	Greeting / who-is-LEX	Fixed intro text	No
2 Refuse	Neither intro nor law	Polite "I can't answer — ask a law question."	No
3+4a Bank hit	Law + match in question bank	LLM rewrite of the verified sheet answer	Yes — structure only
4b Bank miss	Law + nothing in the bank	LLM from general Pakistani legal knowledge	Yes — direct

Urgency words (FIR, court, sue, police, تلاحق...) set `show_lawyer: true` on law answers.

PRODUCT LEX will not quote exact SECP / FBR filing fees. It tells the user to use the platform Fee Calculator. Wire that CTA if the route exists.

6. Length, timing & timeouts

There is one generation cap: `LLM_MAX_TOKENS = 400` (about 1,200–1,800 English characters). There is no separate short/long API. Replies are not streamed — one JSON body when ready.

Reply type	Typical length	Typical time (production)
Short — hello / off-topic	150–350 characters	0.3–0.8 seconds
Long — legal question	800–2,000 characters	2–6 seconds (measured ~2.8s)

Timeout	Value
Frontend abort (recommended)	45–60 seconds
Backend LLM generation	60 seconds
Vercel Hobby hard stop	~10 seconds if the isolate is slow — show Retry

7. Chat history — `session_key` + `localStorage`

On live Vercel, LEX does not persist threads on the server. Keep history in `localStorage`. Still send one `session_key` per thread:

- Same `session_key` for the whole conversation.
- Append the user bubble + response locally after each 200.
- Title the thread from the first message (trim to ~50 characters).

Use the same `session_key` for the whole thread. A new key = a new conversation (blank memory + new sidebar item).

7.1 List threads (sidebar)

```
GET {VITE_LEX_API_BASE_URL}/sessions/
```

```
[
  {
    "id": 1723280000000,
    "session_key": "session_1723280000000_ab12",
    "title": "How do I register a company...",
    "created_at": "2026-08-10T11:02:01.123456+00:00",
    "messages": []
  }
]
```

List items always have `messages: []`. Load the thread with the detail call. Privacy: the list is not filtered by logged-in user. Do not render the full array as “My chats”. Keep keys this browser created and only fetch those.

7.2 Load one thread

```
GET {VITE_LEX_API_BASE_URL}/sessions/:session_key/
```

```
{
  "session_key": "session_1723280000000_ab12",
  "title": "How do I register a company...",
  "messages": [
    { "id": "u_12", "sender": "user", "text": "How do I register a company in Pakistan?" },
    {
      "id": "l_12", "sender": "lex",
      "text": "To register a private limited company...",
      "showReferral": false,
      "referralLabel": "Find a Lawyer →",
      "referralType": "lawyer"
    }
  ]
}
```

Field	Frontend use
sender	user lex
text	Bubble copy
showReferral	Same as chat show_lawyer
referralLabel	Ready-made CTA (EN / UR)
referralType	lawyer → /find-a-lawyer

Unknown key → 404 { "error": "Session not found" }.

7.3 Suggested UI flow

Open LEX

- Read localStorage.lexSessionKeys[]
- For each key → GET /sessions/:key/ (skip 404s)
- Sidebar = titles; click = render messages[]

Send message

- POST /chat/ { message, session_key }
- Append user + lex bubbles from the 200 body
- Save the updated thread in localStorage

New chat → mint a new session_key

HISTORY Client widget only. New chat = POST /sessions/. Sidebar = GET /sessions/. Persist owner_key or send a client JWT.

8. UI contract & errors

- Disclaimer (always): “LEX provides general legal information for Pakistan. It is not a substitute for a licensed advocate.”
- RTL: Language === "UR" on that bubble only (user EN + bot UR can mix).
- Find a Lawyer labels: EN “Find a Lawyer →” · UR “← سیرک شالٹ لی کو”.
- Off-topic: render response as-is. Do not auto-retry.
- 400 empty message · 502 LEX down · abort/timeout → “LEX is temporarily unavailable. Try again.”
- Always use the Vercel Main URL above. Do not call any other LEX host.

9. Sample TypeScript

```
const LEX = import.meta.env.VITE_LEX_API_BASE_URL;

export async function askLex(message: string, sessionKey: string, signal?: AbortSignal) {
  const res = await fetch(`${LEX}/chat/`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message, session_key: sessionKey }),
    signal,
  });
  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(err.error || `LEX HTTP ${res.status}`);
  }
  return res.json();
}

const ctrl = new AbortController();
const t = setTimeout(() => ctrl.abort(), 45_000);
try { await askLex(text, sessionKey, ctrl.signal); }
finally { clearTimeout(t); }
```

10. Smoke test & acceptance checklist

No login required for public chat:

```
curl -s https://nexus-lexis-backend-ql8w.vercel.app/api/v1/lex/chat/ \
-H "Content-Type: application/json" \
-d '{"message": "Hello", "session_key": "fe_smoke_1"}'
```

Expect a greeting. Then try a legal question and an off-topic line (what is the weather?). No login required for the public client widget.

#	Check
1	VITE_LEX_API_BASE_URL set (path already includes /api/v1/lex)
2	Widget POSTs .../chat/ with { message, session_key }
3	Typing indicator 0.3–6s; abort ~45s
4	Urdu bubbles RTL
5	show_lawyer opens Find a Lawyer
6	Disclaimer visible under the widget
7	Same session_key for the whole thread (history + follow-up memory)
8	Sidebar = GET /sessions/ with owner_key or client JWT
9	New chat = POST /sessions/
10	No WebSocket in the production build

11. API index

Method	Path	Auth	Notes
POST	/api/v1/lex/chat/	Public / client JWT	Client widget
POST	/api/v1/lex/sessions/	Public / client JWT	New chat
GET	/api/v1/lex/sessions/	Public / client JWT	History sidebar
GET	/api/v1/lex/sessions/:key/	Public / client JWT	Open thread
DELETE	/api/v1/lex/sessions/:key/	Public / client JWT	Delete thread

Deleted / do not use

Method	Path	Why
WS	/api/lex/ws	Local dev only — Vercel has no WebSockets

HANDOFF Questions: backend repo NexusLexisBackend · contact@nexuslexis.law. Companion docs: Frontend_Production_Handoff · Frontend_Google_and_MainAPI_Flow.